

UNIVERSIDAD NACIONAL JORGE BASADRE GROHMANN
FACULTAD DE INGENIERÍA

ESCUELA DE INGENIERÍA EN INFORMÁTICA Y SISTEMAS



SISTEMAS OPERATIVOS

Informe 4

“Gestión de Hilos”

Nombre y Apellidos:

Willian Arturo Salazar Quispe

Tacna- Perú

2025

Procedimiento

- Creacion y gestion de hilos

```
● arturo@arturo-VirtualBox:~/practica-4$ gcc -o hilos hilos.c -lpthread
● arturo@arturo-VirtualBox:~/practica-4$ ./hilos
Hola desde el hilo 1
Hola desde el hilo 2
Finalizo la ejecucion del programa principal
● arturo@arturo-VirtualBox:~/practica-4$ time ./hilos
Hola desde el hilo 1
Hola desde el hilo 2
Finalizo la ejecucion del programa principal

real    0m0,030s
user    0m0,001s
sys     0m0,028s
```

Se muestra la ejecución y salida del primer programa donde se crean 2 hilos mediante `pthread_create`. Se imprime la salida de ambos hilos y finaliza el hilo principal. La medición mediante el comando `time` refleja un tiempo de ejecución real de 0m0,030s.

- Implementación de sincronización

```
sys    0m0,028s
● arturo@arturo-VirtualBox:~/practica-4$ gcc -o hilos hilos.c -lpthread
● arturo@arturo-VirtualBox:~/practica-4$ time ./hilos
Hola desde el hilo 1
Hola desde el hilo 2
Finalizo la ejecucion del programa principal

real    0m0,006s
user    0m0,004s
sys     0m0,003s
```

Se muestra la compilación y ejecución del segundo programa que incorpora exclusión mutua (`pthread_mutex_t`). Se observa una ejecución ordenada en el acceso a la sección crítica de salida, registrando un tiempo real de 0m0,006s.

- Análisis de Rendimiento

```
● arturo@arturo-VirtualBox:~/practica-4$ gcc -o hilos hilos.c -lpthread
● arturo@arturo-VirtualBox:~/practica-4$ time ./hilos
Ejecutando con 1 hilo
Hola desde el hilo 0
Tiempo total de ejecucion con 1 hilos: 0.005530 segundos
Ejecutando con 5 hilos
Hola desde el hilo 0
Hola desde el hilo 2
Hola desde el hilo 1
Hola desde el hilo 3
Hola desde el hilo 4
Tiempo total de ejecucion con 5 hilos: 0.020255 segundos
Ejecutando con 10 hilos
Hola desde el hilo 1
Hola desde el hilo 2
Hola desde el hilo 5
Hola desde el hilo 6
Hola desde el hilo 7
Hola desde el hilo 4
Hola desde el hilo 8
Hola desde el hilo 9
Hola desde el hilo 0
Hola desde el hilo 3
Tiempo total de ejecucion con 10 hilos: 0.028027 segundos
Finalizo la ejecucion del programa principal

real    0m0.081s
user    0m0.013s
sys     0m0.067s
```

Se muestra la ejecución del tercer programa donde se mide con `<time.h>` el tiempo requerido al escalar el número de hilos. Se evidencia que el orden no está determinado en la impresión de mensajes, por ejemplo, el hilo 0 se imprime al final en la prueba de 10 hilos. Además, se refleja el incremento progresivo en los tiempos tomados (0.005530s, 0.020255s y 0.028027s).

Análisis de Resultados

- **Al comparar el rendimiento de un programa multihilo con un monohilo, ¿qué diferencias observaste en el tiempo de ejecución? ¿Cómo influye la gestión adecuada de hilos en el rendimiento general del programa?**

En el caso de los primeros programas (monohilo) las tareas eran livianas porque usaba el printf para la salida por pantalla. La versión monohilo suele registrar un tiempo de ejecución mínimo como 0.03s debido a la ausencia de sobrecarga

Por otro lado, en el último programa con la ejecución multihilo, se observa que el tiempo total de ejecución no disminuye, en cambio, con tareas más exigidas suele incrementarse, como se observó en la imagen llegando a tener un tiempo de 0.81s. Esto sucede porque la creación de hilos y la sincronización a través de mutex consumen ciclos de CPU adicionales.

En la influencia de la gestión adecuada de hilos, la correcta gestión determina la eficiencia global del sistema. De esta manera, sincronizar correctamente los hilos garantiza que los recursos del sistema operativo se liberen de manera adecuada y que el uso equilibrado de mecanismos de sincronización pueda evitar posibles cuellos de botella.

- **¿Qué funciones de la biblioteca pthread utilizaste en tu implementación? Explica brevemente la finalidad de al menos tres de estas funciones y su importancia en la gestión de hilos.**

Se utilizaron las siguientes funciones

1. pthread_create
2. pthread_join
3. pthread_exit
4. pthread_mutex_init
5. pthread_mutex_lock
6. pthread_mutex_unlock
7. pthread_mutex_destroy

Explicación de tres funciones clave y su importancia

- pthread_create(&hilo, NULL, funcion, arg)
 - Crea un nuevo hilo de ejecución independiente que comienza a ejecutar la función especificada pasando arg como argumento.
 - Es la función fundamental para iniciar la ejecución concurrente o paralela dentro del proceso
- pthread_join(hilo, NULL)
 - Bloquea al hilo llamador hasta que el hilo especificado finalice su ejecución.
 - Permite la sincronización de finalización, evitando que el programa principal termine mientras los hilos secundarios aún están trabajando

- `pthread_exit(NULL)`
 - Finaliza el hilo que la invoca sin necesidad de esperar a que llegue al final de su función principal
 - Garantiza que solo muera el hilo actual, dejando a los demás hilos en ejecución.

- **¿Cómo se comportó el programa al aumentar el número de hilos más allá de un cierto punto?**

Al incrementar el número de hilos, se observa que el tiempo de ejecución no disminuye. En la práctica realizada con el tercer programa, el uso de la biblioteca `<time.h>` permitió obtener los datos del tiempo de ejecución.

Los tiempos registrados en las pruebas fueron los siguientes:

- Hilos 1: 0.005530s
- Hilos 5: 0.020255s
- Hilos 10: 0.028027

Estos resultados demuestran una relación directamente proporcional: a mayor cantidad de hilos, mayor es el tiempo total de ejecución. Este comportamiento se debe a que la tarea ejecutada dentro del hilo está fuertemente acoplada a un mutex. Al aumentar la cantidad de hilos, la mayor parte de ellos permanece bloqueada esperando obtener acceso al recurso compartido en lugar de realizar trabajo útil. De este modo, la sección crítica se convierte en un cuello de botella secuencial.

- **¿Hubo un límite en el que el rendimiento no mejoró o incluso disminuyó?**

Sí, el rendimiento disminuyó progresivamente al aumentar el número de hilos. Esto ocurre porque la tarea dentro del hilo consiste únicamente en acceder a la salida estándar protegida por un mutex. Al incrementar la cantidad de hilos, la contención de la exclusión mutua se vuelve crítica, la mayoría de hilos pasa más tiempo bloqueada esperando adquirir el mutex que realizando cómputo útil.

- **Durante la ejecución, ¿observaste alguna condición de carrera o comportamiento inesperado? Si es así, ¿cómo impacto esto en los resultados obtenidos y que medidas se podrían tomar para prevenirlo?**

No se produjeron condiciones de carrera gracias al uso de mutex, pero sí se observó un comportamiento no determinista en el orden de ejecución. Esto se evidencia en la ejecución del último programa, en donde los hilos no terminan ni imprimen en el orden exacto en que fueron creados. Esto es un comportamiento normal en entornos concurrentes

Para prevenir condiciones de carrera o comportamientos inesperados se tiene las siguientes medidas:

- Emplear algoritmos de exclusión mutua (`pthread_mutex_t`), semáforos o variables de condición para asegurar el acceso atómico a la sección crítica.

- Minimizar el código que requiere bloqueo para reducir la contención entre hilos.
- Evitar el uso innecesario de variables globales compartidas.